

Breaking the Bottleneck: Attention-Based LSTMs for Grapheme-to-Phoneme Translation

Paul Kabulu
KBLPAU002

18/05/2026

1 Introduction

Encoder-decoder LSTM models are a standard choice for sequence-to-sequence tasks like machine translation and text-to-speech. However, they often suffer from an information bottleneck, as the entire input sequence must be compressed into a single, fixed-size vector.

This report evaluates three different decoder configurations for the Grapheme-to-Phoneme (G2P) task. The primary goal is to determine whether attention mechanisms offer a tangible performance boost over standard, fixed-context approaches.

2 Architecture Design

All three models share identical encoder and embedding layers. They differ only in how the decoder accesses the encoder’s internal representations.

Setup 1: Bottleneck Baseline (No Context)

The encoder produces a final hidden state (h_{enc}, c_{enc}) which is used to initialise the decoder. After that, the decoder receives no further information from the encoder. This means all input information is squeezed into a single vector, which inherently limits the model’s ability to handle longer or more complex words.

Setup 2: Fixed Context Vector

In this version, the final encoder hidden state $z = h_{enc}$ is passed into every decoding step. The LSTM gates become:

$$\begin{aligned}f_t &= \sigma(W_f y_t + U_f h_{t-1} + V_f z) \\i_t &= \sigma(W_i y_t + U_i h_{t-1} + V_i z) \\o_t &= \sigma(W_o y_t + U_o h_{t-1} + V_o z) \\\tilde{c}_t &= \tanh(W_c y_t + U_c h_{t-1} + V_c z)\end{aligned}$$

This provides the decoder with continuous access to a global summary of the input sequence. While this slight modification helps, the provided context remains static across all time steps.

Setup 3: Dynamic Attention

Here, the model computes a different context vector at each time step using dot-product attention:

$$\begin{aligned}e_{t,j} &= h_t^T h_{enc,j} \\ \alpha_t &= \text{softmax}(e_t) \\ z_t &= \sum_j \alpha_{t,j} h_{enc,j}\end{aligned}$$

By replacing the fixed context with a dynamic one, the decoder can adaptively focus on relevant parts of the input sequence based on its current prediction state.

Implementation Note: EOS Truncation

During batch decoding, sequences naturally finish at different times once an EOS token is generated. To ensure consistent Phoneme Error Rate (PER) calculations, predictions are truncated at the first EOS token so that padding and leftover artifacts don't skew the evaluation.

3 Hyperparameter Tuning

I conducted a grid search on Setup 1 using 27 combinations (3 learning rates $\times$ 3 embedding sizes $\times$ 3 hidden sizes). Due to time constraints, each model was trained for 5 epochs.

Learning Rate	Embedding Size	Hidden Size	Validation PER
1e-5	128	256	0.6483
1e-4	128	256	0.2576
1e-3	64	256	0.1373

Chosen configuration: $lr = 1\text{e-}3$, embedding = 64, hidden = 256.

Reasoning:

- A learning rate of 1e-3 gave the best results and converged noticeably faster.
- An embedding size of 64 provided the best balance, 32 resulted in underfitting, while 128 offered no additional performance gains.
- A hidden size of 256 consistently performed best, indicating that a larger model capacity helps retain sequence information despite the bottleneck.

4 Experimental Results

All models were trained under identical conditions (same seed, batch size, and early stopping with patience = 5).

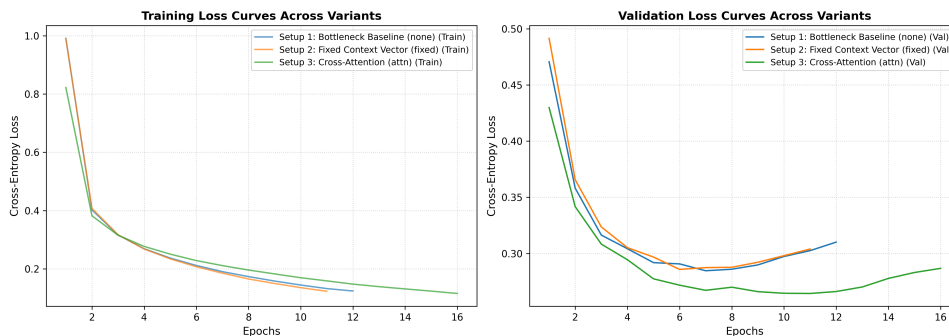
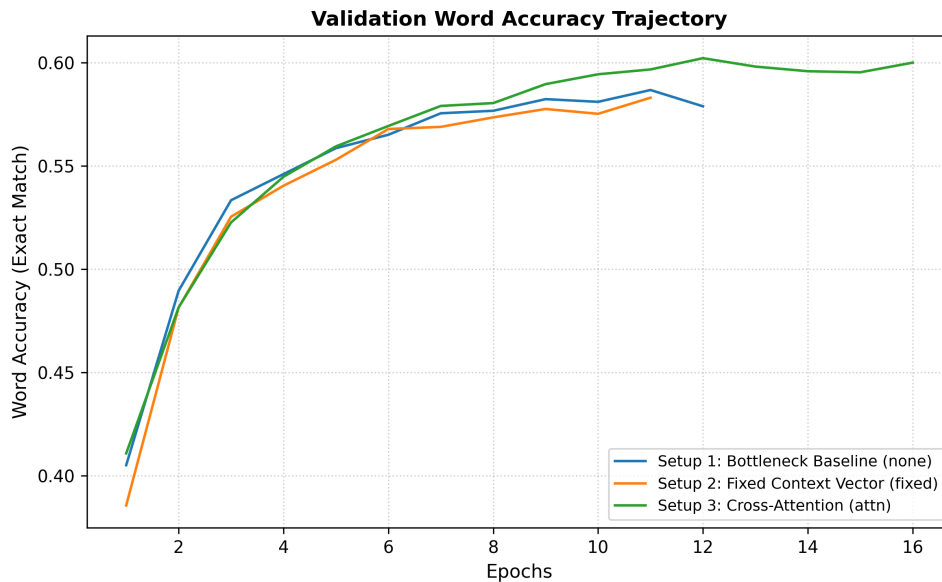
Training and Validation Behaviour

The training loss trajectory was nearly identical across all models, dropping from roughly 1.0 to ~ 0.1 . This confirms that all configurations had adequate capacity to learn the training data. The key differences emerged in validation performance:

- **Setup 1:** Improves rapidly early on but begins to overfit at $\sim$ epoch 7.

- **Setup 2:** Performs slightly better but displays a similar overfitting pattern.
- **Setup 3:** Continues to improve for longer, ultimately reaching the lowest validation loss.

Validation accuracy follows a similar trend, with Setup 3 peaking around 60%, while the others plateau around 57–58%.



Test Set Performance

Setup	PER	Word Accuracy
Setup 1 (no context)	0.1352	57.21%
Setup 2 (fixed context)	0.1326	58.10%
Setup 3 (attention)	0.1173	59.91%

Setup 2 provides only a marginal improvement over the baseline. Setup 3 delivers the clearest gain, achieving the lowest error rate and the best overall accuracy.

5 Why Attention Helps

The primary flaw in Setup 1 is the forced compression of the entire input into a single vector, ultimately discarding detail for longer and more complex words. Setup 2 mitigates this slightly

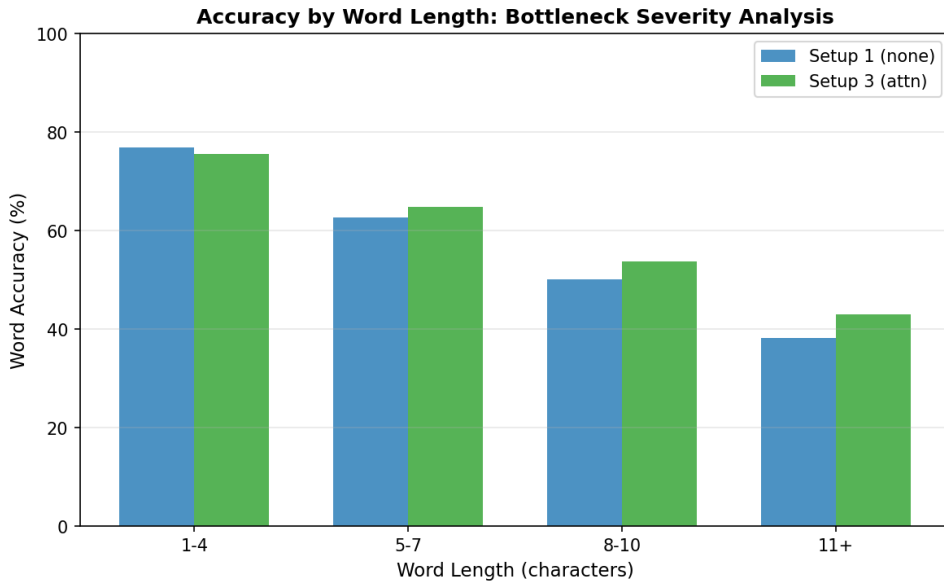
by continually referencing that same context vector, but its static nature means it cannot adapt to the decoder’s ongoing generation. Setup 3 resolves this by recalculating the context at every step, allowing the model to shift its focus across the input sequence as needed. This effectively bypasses the severe information bottleneck.

6 Character Lengths

To analyze the impact of word length on performance, the evaluation results were stratified by character count.

Length	Setup 1	Setup 2	Setup 3	Improvement (1 → 3)
1–4 chars	76.9%	75.4%	75.5%	-1.4%
5–7 chars	62.7%	64.4%	64.9%	+2.2%
8–10 chars	50.6%	50.0%	53.8%	+3.7%
11+ chars	38.1%	37.4%	43.0%	+4.9%

The results highlight a clear trend: Setup 1 (the baseline bottleneck) performs significantly worse on longer words, falling from 76.9% accuracy on short words to 38.1% on words with 11+ characters—a steep 38.8% drop. Setup 3 (with attention) also declines, but less sharply, dropping from 75.5% to 43.0% (a 32.5% drop). This narrower performance gap suggests that attention actively helps mitigate the bottleneck problem for longer sequences. Ultimately, attention provides minimal improvement on short words (-1.4%) but yields much larger gains on long words (+4.9%), where the translation task is considerably harder.



7 Conclusion

Overall, the experimental results shows that dynamic attention mechanisms improve the performance of encoder-decoder LSTMs on G2P tasks.

- The baseline model struggles significantly with longer inputs.
- A fixed context vector offers minor improvements but remains structurally limited.

- The attention-based model consistently performs best, particularly on more complex cases.

While the absolute accuracy improvements are relatively small, they are highly consistent and align perfectly with the theoretical benefits of reducing the information bottleneck in sequence-to-sequence architectures.